Annals of Telecommunications (2024) 79:803–817
https://doi.org/10.1007/s12243-024-01018-4

Mirage: cyber deception against autonomous cyber attacks in emulation and simulation

Michael Kouremetis[1] · Dean Lawrence[1] · Ron Alford[1] · Zoe Cheuvront[1] · David Davila[1] · Benjamin Geyer[1] · Trevor Haigh[1] · Ethan Michalak[1] · Rachel Murphy[1] · Gianpaolo Russo[1]

Received: 19 December 2023 / Accepted: 23 February 2024 / Published online: 13 March 2024

Abstract
As the capabilities of cyber adversaries continue to evolve, now in parallel to the explosion of maturing and publicly-available artificial intelligence (AI) technologies, cyber defenders may reasonably wonder when cyber adversaries will begin to also field these AI technologies. In this regard, some promising (read: scary) areas of AI for cyber attack capabilities are search, automated planning, and reinforcement learning. As such, one possible defensive mechanism against future AI-enabled adversaries is that of cyber deception. To that end, in this work, we present and evaluate Mirage, an experimentation system demonstrated in both emulation and simulation forms that allows for the implementation and testing of novel cyber deceptions designed to counter cyber adversaries that use AI search and planning capabilities.

Keywords Cyber security · Deception · Adversary emulation · Reinforcement learning

1 Introduction

The evolution of cyber adversary attack capabilities is on display nearly every day and has become a part of the new normal for cyber defenders, stakeholders, and society at large. So much has this become the accepted paradigm that cyber defense is conducted in the planning and operational model of a permanent arms race with cyber adversaries [18, 31]. One notable domain of potential evolution is that of using artificial intelligence (AI) technologies and advancements for cyber attacks [16, 19, 41, 44].

While once thought to be impossible or simply decades away, in 2016, enhancements in search and neural network classifiers as well as computing advancements led to the creation of the AlphaGo system, capable of decidedly outperforming the world's best Go players [38]. As such, observers in many domains may reasonably wonder, if a game with a branching factor of 250 can be conquered, how long until other "impossible", complex problems are also dethroned. Within the cyber domain, one can already observe efforts and advancements in the ability to carry out autonomous cyber attacks [2, 4, 16, 21]. While the equivalent of an AlphaGo-level autonomous cyber attack system has not yet presented itself, one can objectively assert that cyber defenders no longer believe such an adversary is abstract [7, 41].

Consequently, defense against an AI-enabled, autonomous adversary will require cyber defense technology, tactics, and strategies that specifically counter the competitive advantages gained from an adversary's use of AI. Along these lines, one promising cyber defense tactic is that of cyber

✉ Michael Kouremetis
mkouremetis@mitre.org

Dean Lawrence
dlawrence@mitre.org

Ron Alford
ralford@mitre.org

Zoe Cheuvront
zcheuvront@mitre.org

David Davila
ddavila@mitre.org

Benjamin Geyer
bgeyer@mitre.org

Trevor Haigh
thaigh@mitre.org

Ethan Michalak
emichalak@mitre.org

Rachel Murphy
rmurphy@mitre.org

Gianpaolo Russo
grusso@mitre.org

[1] The MITRE Corporation, McLean, VA, USA

deception. Cyber deception can deflect, distort, deplete, and discover cyber adversaries and attacks, and be tailored to interact or engage with adversaries to achieve precise detrimental effects [1].

In this work, we envision (and implement) a future cyber adversary whose actions and decisions are entirely controlled by an autonomous system. This autonomous system uses search techniques to drive its cyber attack operations and achieve the desired objective(s). Given such a scenario, we ask the questions of (1) whether novel cyber deceptions can be constructed and deployed in such a manner as to directly target weaknesses in the automated planning and search techniques; and (2) can an effective emulation system be implemented to evaluate cyber deceptions and autonomous adversaries against each other at scale. To answer and explore these questions, we present and evaluate Mirage, a cyber deception and autonomous cyber adversary experimentation system.

This article is an extension of previously published work [22], extending the work with an analysis of the robustness of our cyber deceptions against a reinforcement learning agent in simulation.

1.1 Key contributions

- Three novel cyber deceptions purpose-built for countering autonomous cyber attacks.
- Anansi - a Windows operating system deception service framework to deploy and actuate cyber deceptions.
- Experimental analysis for 72 offensive cyber operation trials in emulation, varying the adversary profile, choice of planning technique, and employed deceptions.
- Demonstration of the robustness of deception, evaluating it in a high fidelity cyber simulation environment against reinforcement learning agents.

2 Background

2.1 Cyber deception

Cyber deception is generally described as any planned action(s) taken to mislead and/or confuse attackers, thus causing attackers to take specific actions that aid in the actual defense of the cyber system under which the deception is being applied to [43]. Cyber deception has long found significant application and value within computer systems and networks [12, 32]. Common cyber deception constructs include honeypots [30] and honeytokens [13], which are fake/decoy computer systems, resources, and/or data that are deployed on real computer and network systems for purposeful influence and engagement with an attacker.

For the purposes of this work, the cyber deceptions created and evaluated within Mirage can all be categorized as novel honeytoken implementations. Specifically, Mirage's cyber deceptions are all computer file-based cyber deceptions.

2.2 Adversary emulation

Adversary emulation is a variant of the discipline of cyber red teaming that aims to emulate a known cyber threat/adversary to much higher fidelity with regard to known actions, behaviors, and objectives than standard red teaming would normally account for [3]. As a sub-discipline, adversary emulation was established to address the needs of evaluating whether computer systems and networks were protected against specific advanced persistent threats (APTs). In effect, it is a marriage of the discipline of cyber threat intelligence to the red teaming process. While adversary emulation is not the direct focus of our work, it nevertheless provides the tools and infrastructure for our experimentation program. For our work, our chosen adversary emulation platform is Caldera [28].[1]

2.3 Reinforcement learning

Reinforcement learning (RL) is a subset of machine learning that focuses on training agents, which can be computer programs or physical robots, to make decisions in dynamic environments. The central idea behind RL is that the agent learns by interacting with its surroundings through a simulated environment often called a "gym" [5]. Instead of being provided with labeled data as found in classical machine learning algorithms, the agent generates its own experience data to learn from. The RL agent explores its environment using a decision-making policy to observe the current state and take actions it decides are optimal. After each action, the agent receives a reward from the environment which serves as feedback, indicating how good or bad the action was in terms of achieving the agent's goals. The primary objective of the RL agent is to improve its decision-making policy to maximize the cumulative reward it can accumulate over time. It aims to learn to associate specific actions with higher rewards resulting in a policy that consistently selects actions that lead to higher rewards.

There are a variety of training algorithms that can be used to derive and optimize the policy. Policy-based approaches such as Proximal Policy Optimization (PPO) attempt to learn the policy directly. In contrast, value-based approaches like Q-learning first attempt to find an optimal value function which estimates the quality of each state-action pair, and then derive the policy from the value function [25]. Policy-based

[1] Some of the authors of this work also hold dual-roles on the Caldera project team.

algorithms are generally less sample-efficient but are better suited for stochastic environments and continuous action spaces, whereas value-based algorithms are more sample-efficient but are limited to discrete action spaces and only optimize the policy indirectly. Actor-Critic methods combine elements of both types by having an actor learn the policy and a critic estimate the value function. Recent approaches such as Asynchronous Advantage Actor-Critic (A3C) have shown considerable performance with regard to training speed but still have some drawbacks such as instability and increased implementation complexity.

All of the above methods fall within the category of model-free algorithms, where experiences are solely derived from interactions with the environment. In contrast, model-based methods train an additional internal model of the environment used for simulating future scenarios. The policy can plan ahead and train on these model-simulated trajectories. Model-based approaches can work very well for problems with well-defined environment rules such as in physics-based simulations. Nevertheless, they are not without their drawbacks, including limitations imposed by the trained model, which can include bias from model-approximation errors, limited data, and structural assumptions about the real environment [25].

3 Related work

For the purposes of our work in Mirage, related research and development work can be categorized into the overlapping areas of cyber attack simulation and emulation environments, cyber deception deployment systems, and cyber deception research. Recent work in these areas is detailed.

Within the past few years, due to the growth and accessibility of RL "gyms," there have been multiple independent efforts focused on cyber attack gyms with varying levels of fidelity and scenarios of focus. CyberBattleSim is a lower fidelity/high-level RL gym for training an attacker agent against a simulated cyber network environment [27]. CyberBattleSim focuses on a limited set of attacker actions, such as lateral movement. Walter et al. extended CyberBattleSim with the deception components of decoys, honeypots, and honeytokens in order to evaluate the optimal placement of deception components against attacker agents [42]. Similarly, NASim is another RL gym using a simulated cyber network environment to evaluate the effectiveness of deceptions, specifically attempting to answer defensive value questions such as how many honeypots to use against certain cyber attacker models [33]. CybORG is a gym focused on the training of defensive agents given specified cyber network scenarios (environments) and attackers (adversarial agent) [40]. CybORG has higher fidelity/low-level action space for its simulation environment, making it closer to realistic cyber attack scenarios than CyberBattleSim. The first CybORG challenge included both simulation and emulation operations, but subsequent challenges are pure simulation. CyGIL is an RL gym built to work with an emulated environment instead of a simulated environment [24]. In this regard, CyGIL is quite novel and allows for an extremely high fidelity/low-level action within its RL gym. Both CyGIL and Mirage use Caldera for their attacking agent and offensive action space.

Beyond simulation and emulation environments, research has also been conducted in developing applied cyber deception systems. Al Shaer et al. developed automated cyber deception design and deployment systems, including DodgeTron [34], CHIMERA [17], and SODA [35]. These deception systems all center around the automation, deployment, and efficacy of cyber deceptions against targeted malware instances. Specifically, CHIMERA and SODA focus on the targeting and actuation of deceptions towards adversarial tactics and behaviors gleaned from previously analyzed malware samples. Additionally, at the concept and ontology level is the MITRE Engage framework which is a knowledge base for cyber deceptions and adversary engagement [29].

Lastly, there is notable work on the quantification of the effects of cyber deception on cyber adversaries. Ferguson-Walter et al. conducted multiple, significant studies on the effects of cyber deception on actual cyber attackers by way of large human research trials with penetration testers, red-teamers, and computer specialists [10, 11, 37]. These studies are entirely centered around human adversaries but do conclusively quantify the detrimental effect of cyber deception on a cyber attacker. Similarly, these effects have also been cataloged from a concept and case study perspective in [8].

4 Emulation

To quantify the effects of file-based deception against automated adversaries, we designed and built a framework for deception experimentation that runs in *emulation* — on virtual machines running standard commercial software. In the remainder of the section, we will discuss the details of the framework and the results of our experiments.

4.1 Experimentation framework

Each trial in our experiments consists of a specific adversary and deception choice. These are deployed together on a small Windows domain running in an Amazon Web Services environment. These components are shown in Fig. 1, and described below.

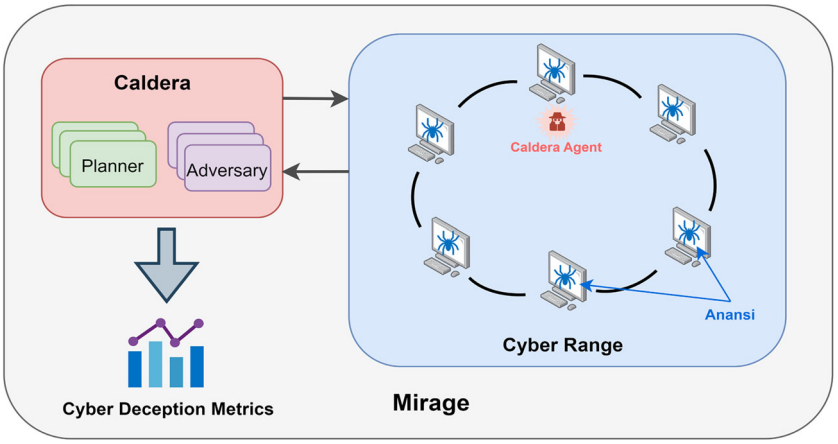

Fig. 1 The operational architecture of Mirage emulation system

4.1.1 Adversary profiles

Two adversary profiles were defined to evaluate deceptions against. Both target Windows systems and were implemented in PowerShell. A full list of abilities can be found in Appendix A.

Thief. A simple exfiltration adversary that contains Tactics, Techniques, and Procedures (TTPs) for creating a staging directory, performing sensitive file discovery, copying the found files to the staging directory, compressing the staged files, and finally exfiltrating the zipped archive back to the Caldera server.

BlackSun. A ransomware adversary that contains a diverse set of TTPs aimed at both finding and encrypting sensitive files, as well as covering its tracks. This adversary was modeled on a real-world threat of the same name first observed in 2020, written in PowerShell for access to Microsoft cryptography modules [6].[2]

Both adversaries contain a matching set of lateral movement abilities targeting a Windows environment. At a high level, these abilities discover user credentials, use those credentials to open a file share to a remote host, copy the agent binary to the remote host over the file share, and then executing the binary using Windows Management Instrumentation (WMI). This process creates an additional Caldera agent on the new host for the server to make use of when choosing actions.

4.1.2 Cyber attack planners

The core Caldera platform provides the main capabilities for matching facts in its knowledge base against the requirements for abilities in an adversary profile to determine which actions it is able to take. The platform also allows for more complex decision-making logic through Caldera modules called *planners*. These planners are able to overlay any decision logic over the actions currently available to the adversary profile and further decide which specific actions to take.

The following planners were selected for use in our experimentation program:

- **Batch** - A simple planner that executes all available actions at each iteration. The planner is primarily used as a base line in the experiments.
- **Look-Ahead** [14] - Chooses a single action at each iteration based on the expected reward. Action-reward values are set by the user apriori. Then in the operation, the planner calculates rewards for abilities based on the discounted values of ability sequences up to a maximum depth.
- **Guided** [23] - Constructs a directed attack graph and performs a goal-based search to find and execute actions that lie along the shortest path to the goal. At each iteration, the planner chooses the action closest to its goal.[3]

4.1.3 Cyber deceptions and deception service

As the primary component under test, the cyber deceptions were the target of our Mirage experiments. To create novel cyber deceptions specifically tailored towards inhibiting an autonomous cyber adversary/attack, we first identified general planning techniques that are found across artificial intelligent search and planning algorithms and consequently focused on three of those techniques. In effect, we make the

[2] The Caldera BlackSun adversary is currently closed-source.

[3] The Batch and Look-Ahead planners were already present in Caldera's open-source repositories [28] but the Guided planner was a novel contribution, created and open-sourced during the course of this work.

strong assumption that any autonomous cyber attack system will use the following techniques in their implementation:

- Attempt to reduce state space via (1) ignoring or abstracting state space, (2) removing state space via heuristics, sub-goal localization, and (3) removing symmetric branches/paths.
- Will do online planning and decision-making; that is will have the ability to re-plan.
- Operations will be goal-oriented, which will fall inline with common cyber attack objectives (e.g., persistence and data theft)

Given these assumed search and planning techniques (in use by the adversary), we proceed to develop deceptions that directly exploit them for negative effects. For example, if an adversarial autonomous cyber attack system aims to reduce the state space, the goal of our novel deception is to purposely prevent such state space reduction or even expand it. Table 1 details the three novel deceptions created for evaluation against an autonomous adversary.[4]

An observer will note that these deceptions are all based around computer host file objects. This was due primarily to the complexity of deception implementation — manipulating file operations are relatively simpler than more layered cyber deception. However, file-based deceptions are also currently one of the most commonly deployed and effective cyber deceptions on real-world systems [45].

As part of our simulation infrastructure, there was a need for a deployable, dynamic deception framework that would change known states of a planner's environment in the form of adversary deception. As such, we created *Anansi*.

Anansi works by monitoring PowerShell log files for all users that appear locally on the machine and examining each command for keywords that are the trigger signature for a deception to be deployed, as a live reaction to adversary activity. Deception configurations and trigger signatures are kept in a JSON file placed on endpoints during deployment. The Sneaky Files and Black Hole deceptions were implemented and deployed with *Anansi*. For more information on how these deceptions operate (please see Table 1).

4.1.4 Cyber range

For Mirage's cyber range infrastructure, we use Caldera Range, a closed-source plugin for the Caldera platform that allows for Caldera to seamlessly connect to a cyber range environment, deploy agents (implants), and execute operations. We used the Amazon Web Services (AWS) cloud infrastructure backend for the tool while executing on our experimentation program. The targeted network for our experiments consisted of five Windows host machines and a single Windows domain controller.

Each host in the domain was seeded with user credentials for a single neighboring host in a ring topology. Each host was also populated with six sensitive files, split between two directories. Each file was initialized with a random filename. One host began with a Caldera agent running on it to serve as a starting point for the operation.

4.1.5 Experimentation program and metrics

The suite of experiments consisted of three episodes per combination of adversary profile, cyber attack planner, and deception strategy. Using the two adversary profiles, three cyber attack planners, and three deception strategies outlined in previous sections, this resulted in a total of 72 cyber operations completed over the experimentation program. The total number of episodes per combination was limited to three due to the time-consuming nature of the process for initializing the cloud environment, performing the operation, and tearing down the environment.

Caldera produces a detailed operation report as a JSON file upon the completion of an operation. These operation reports were used to compute a set of metrics for answering analytical questions concerning the performance of the adversary against the various deception strategies. Descriptions of the metrics computed from the operation reports are as follows:

- Total number of actions executed over the course of the experiment.
- Number of actions that failed to complete.
- Number of actions that were repeated multiple times in the experiment.
- Time spent on failed actions in seconds.
- Time spent planning choice of next actions.
- Number of facts learned over each trial
- Cumulative score over all learned facts.
- Total experiment run-time in seconds.

4.2 Experimentation results

The first goal of this work was to test novel cyber deceptions against autonomous cyber adversaries and evaluate the effects of the deceptions on those adversaries. The following sections detail the overall performance of the deceptions and discuss the validity of the chosen evaluation metrics.

[4] To our knowledge, these deceptions (both concept and implementation) are novel, except for the concept of the File Facade deception which is found in previous work [39].

Table 1 Descriptions of novel cyber deceptions for targeting autonomous cyber attack systems

Deception	Description	Targeted Planning / Search Technique
Black Hole Directory	Any attempt at file collection by the adversarial agent results in the exfil directory being targeted and all files moved out of the directory. This produces a latent effect on the agent as the lack of files in the exfil directory will not be discovered until exfiltration is attempted	Incorrect belief state; Unproductive journey
File Facade	Exfiltrated files are replaced with large, random files	Unproductive journey
Sneaky Files	When an adversarial agent performs file discovery commands, a reactive hook will change the names of all files in specified locations to random UUIDs. This changes the conditions of the environment and alters the facts understood by the agent	Incorrect belief state; inducing agent re-planning

4.2.1 Performance of cyber deceptions

Figure 2 shows the overall experimental results view of the two cyber adversaries (Thief, BlackSun) for each of the novel cyber deceptions detailed in Table 1, over the two chosen metrics of *successful_action_proportion* and *proportion_of_time_spent_on_planning*. As our experimentation program had 8 metrics, 2 adversary profiles, and 3 planners there are 150 specific data views. We chose this data view for the paper as it is an accurate representation of the performance of all deceptions across the 2 adversaries. For this data view, the best-performing deceptions are found in the bottom right and the worst-performing deceptions are in the top left.

Key observations of the experimentation program were as follows:

- All the deceptions had a clear (negative) effect on the adversarial cyber planners, regardless of the adversary profile (i.e., Thief, BlackSun) or planner (i.e., Guided, Look-Ahead). Observing Fig. 2, the data points are nearly linearly separable and even inclined to clustering.
- For the Thief adversary, the deceptions had distinct negative effects on the Guided and Look-Ahead planners.

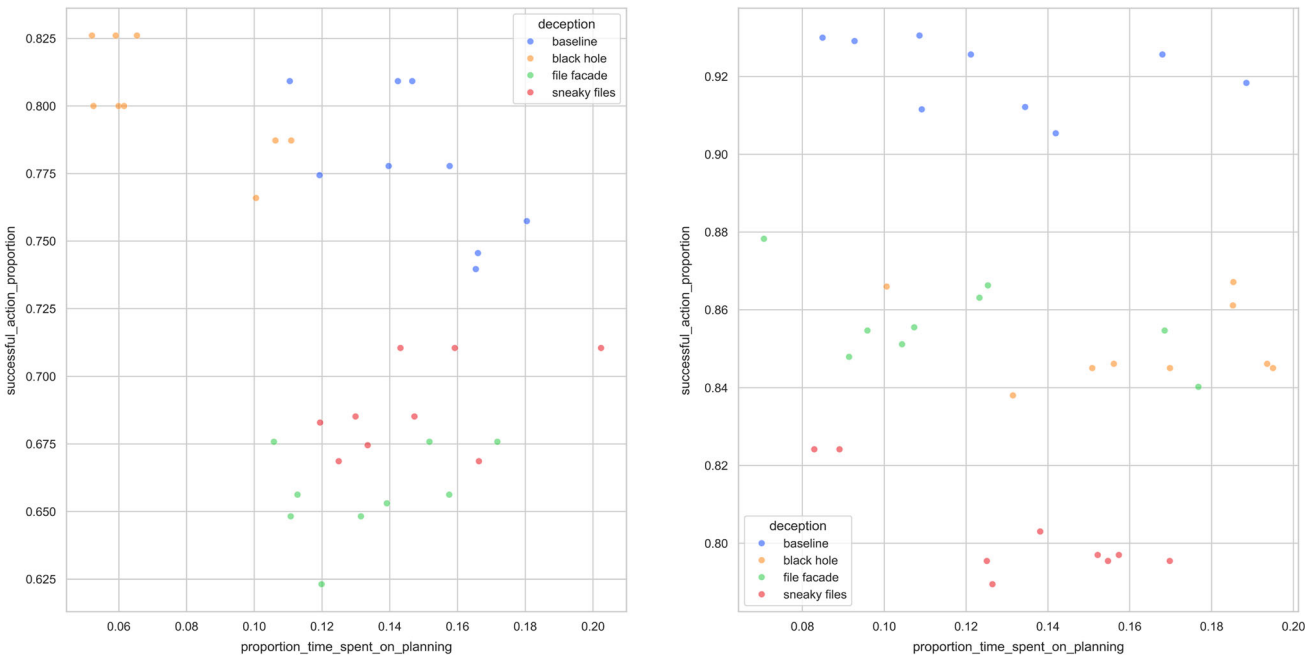

Fig. 2 Mirage experiment results for the Thief adversary (left) and BlackSun ransomware adversary (right), with hue color denoting cyber deception deployed in cyber range. In these charts, the x-axis is the percentage of time the adversary spent on planning, and the y-axis is the percentage of successful actions executed by the adversary

Notably, these planners were faster, but the deceptions induced many more failed actions.
- The File Facade deception forces the Guided and Look-Ahead planners to consider more information and thus take significantly more time when planning.

4.2.2 Cyber deception metrics

As outlined in Section 4.1.5, Mirage captured 8 metrics from the cyber operation experiments, all aimed at capturing the performance of the decision component of the autonomous cyber adversary. We (generally) conclude that these metrics served as appropriate measures of impact to the adversarial cyber planning component as one metric (*proportion_time_spent_on_planning*) was directly related to planning performance and the rest were strong proxies (e.g., *total_actions*, *failed_actions*, *cumulative_score_over_all_learned_facts*, and *total_run_time*). With regard to the proxy metrics, in Mirage's controlled cyber ranges, noise and unpredictability are reduced enough to confidently conclude that action failures or unexpected deviations are the result of Caldera's planner responding to the deceptions, not other external factors. For example, if the Thief adversary takes some additional failed actions not seen in the baseline experiment, these failed actions are not the result of chance or minute changes in the cyber range. Furthermore, our team was able to select random failed actions and manually verify their cause by using Caldera's knowledge base and post-operational report. However, one key metric missing from this work, that is usually found in the evaluation of automated planners, is that of the number of states evaluated by the cyber adversarial planning system for a given action decision. Future experimentation should include the mechanisms to capture internal decision metrics.

4.2.3 Computation time and costs

For the 72 emulation cyber operations, it took 24.76 hours of compute time on six AWS t2.medium instances with each instance costing $0.0644/hour. We estimate that provisioning stand-up and tear-down of the compute resources roughly doubled compute time. The total cost (stand-up, experiment, tear-down) for all experiments was about $19 or about $0.25 per experiment. Total compute time for all experiments was nearly 50 hours or 69 minutes per experiment.

While both the computational cost and time may not be prohibitive, scaling the experiments can quickly become intractable, especially noting that our experiments had a simple flat network. This trend confirmed a key hypothesis that a higher fidelity emulation system of deceptions and autonomous adversaries has physical limits, despite the efficiencies found in automated cyber range deployment and autonomous Caldera execution of operations. These conclusions are inline with previous works [24, 40].

5 Simulation

As noted above, temporal limitations inherent in emulation notably precluded the ability to scale experimentation of cyber deceptions, computer system environments, and autonomously enabled adversaries. To overcome this challenge, our team worked this past year on evolving Mirage from using an emulation system to that of a high-fidelity simulation system. That is, a simulation system and framework characterized by high fidelity, easily definable, and extendable cyber environments; comprehensive cyber attack agents and action spaces; and the ability to overlay and encode cyber deceptions into the environment.

Additionally, with the transition to a simulation environment, the Mirage system was able to expand from not only evaluating automated planning-based adversary cyber agents but also reinforcement learning-based agents. As detailed below, for our initial experimentation of the simulation system, we opted to use reinforcement learning agents, as opposed to automated planning-based agents.

The following section details the simulation system used by Mirage, known as *CyberLayer*, and the reinforcement learning gym our team integrated with CyberLayer. Additionally, we discuss how deception defenses are embedded in the simulated environment and our initial experiments with a reinforcement learning adversarial cyber agent.

5.1 Environment

5.1.1 CyberLayer

CyberLayer is a simulation environment for cyber operations based on the AI gym design pattern. It generally conforms to the pattern of many such reinforcement learning frameworks.[5] The critical requirement was that the environment be highly representative of real-world environments. A learned policy in the CyberLayer environment necessarily needed to transfer and perform just as well in the real-world environment that the environment's computer network model was based on. This required the environment topology to be represented through an accurate data model and underlying data structures. It also required that the action space conform to and provide all the constraints of live-fire cyber actions in actual environments. A learned policy in CyberLayer for a given toolset (translated to a given action space), and a given

[5] CyberLayer is developed by MITRE and is currently closed-source.

environment should perform the same in the simulation as it would a live-fire implementation of the environment.[6]

In order to create an environment with this kind of realism but still at a lighter weight level than emulation or virtualization, significant care was put into the logical evaluation of when a move was valid, the state changes that would be caused by different moves, and the ways that the action space was effected based on an agent's current position.

With the basic foundation of a computer network environment representing computer systems, file systems, and the networks that interconnect these systems, it was fairly straightforward to implement mechanics that would interact with these systems for deception effects. If an action such as "ls" would list files in a directory, when an agent ran "ls" under deception circumstances, before the observations were returned to the agent the deception system could be called to return the outputs of a designated deception strategy (e.g., file obfuscation and additional files) and those could be injected into the observations of the agent. As in real-world deception, this creates circumstances where not all agent observations perceived of the environment correlate with ground truth reality of what is in the environment.

To our knowledge, the previous works of CybORG [40] and CyGIL [24] are the closest analog to CyberLayer. Currently, CybORG is open-sourced, and CyGIL and CyberLayer are closed-source.

Relative to CybORG, CyberLayer operates at a different level of abstraction with regard to action spaces. CybORG's action spaces consist of actions like "Discover Remote Systems," "EscalateAction," and "StopService," which one could say are at the equivalent level of MITRE ATT&CK techniques. Comparatively, CyberLayer's action space consists of direct computer system commands, TTPs, and/or exploits. For example, it has such actions as "net view," "nbtstat," "Get-DomainComputer" commands. Additionally, CybORG was built for large-scale open contests (i.e., CAGE Challenges) where competitors are responsible for building the corresponding agents for the specific CybORG environment and scenario. This differs from CyberLayer in that CyberLayer has both environment and agent components natively.

Relative to CyGIL, CyberLayer takes the more traditional approach of being a direct, manually implemented simulation system, versus the novel emulation-simulation feedback system that CyGIL maintains. In effect, CyberLayer does not require the "burn-in" emulation cycles that CyGIL requires; however, new action spaces and environment effects must be manually coded into CyberLayer, while in CyGIL they can be learned and translated more autonomously. Similarly, CyberLayer and CyGIL both contain action spaces that do contain subsets of actions taken from MITRE Caldera.

[6] Some of the authors of this work also hold dual-roles on the Caldera project team.

5.1.2 Reinforcement learning gym

As the CyberLayer simulation system adheres to the Open AI Gymnasium API, it can be integrated into standard reinforcement learning gyms. Our team chose the open-source reinforcement libraries Ray and RLlib as the gym to integrate the CyberLayer environment into.

Ray and RLlib are two open-source libraries that are commonly used for reinforcement learning (RL) and distributed computing. They work together to provide a powerful framework for training and deploying RL agents in various environments. Ray serves as the underlying distributed computing framework that powers RLlib's distributed training capabilities. It manages the allocation of resources, such as CPUs or GPUs, for training RL agents in parallel. RLlib leverages Ray's task scheduling and distributed data processing features to efficiently distribute RL training workloads across available compute resources. Users can define custom RL environments and experiment configurations using RLlib's high-level APIs. RLlib provides a variety of RL algorithms that can be easily integrated into user projects, and users can experiment with different algorithms to find the best fit for their specific problems [26].

As CyberLayer is the environment of our reinforcement learning model, it interfaces with the RLlib gym framework in two key ways: supplying observations and rewards back to the RLlib agent, and receiving actions from the RLlib agent and executing them in the CyberLayer environment.

For supplying the learning agent with the observation of the environment, Gymnasium *ObservationWrappers* were utilized to wrap the entire CyberLayer environment state and filter down to the desired observation space (and appropriate encoding format) for the agent to receive. CyberLayer is designed to provide access to its comprehensive environment state so that any possible observation space may be used. Thus Gymnasium *ObservationWrappers* could be implemented for any observation space that is desired by merely querying CyberLayer for any state information the observation space requires. Similarly, the reward policies used in the reinforcement learning model are architected to be modular and supplied to the CyberLayer environment at run-time. Reward policies also have access to the comprehensive environment state of CyberLayer, and any policy or function may be calculated over the environment state to get the reward for the current step.

Equivalently, CyberLayer receives actions from the reinforcement learning agent through Gymnasium *ActionWrappers*.

ActionWrappers are used to decode the output from the agent's model/policy as well as execute any furthe preprocessing (e.g., grounding or variable replacement) required before sending the action to CyberLayer to execute. Finally, before the output (chosen action) is passed from the agent to the *ActionWrappers* (and then to the CyberLayer environment), it is also put through RLlib *action masking*. Action masking allows for the clipping (reducing) of the action space available to the agent for the current step, by removing actions that are not allowed or irrelevant in the environment. Our team found that using action masking in our initial experiments can greatly improve training, and/or more quickly identify optimum training hyperparameters.

Figure 3 shows the reinforcement learning gym and CyberLayer environment used by the Mirage system.

5.2 Agent design

The setup of the reinforcement learning game makes the assumption that agent choice of action occurs only locally to a host on a network. This means that in an operational environment, each agent would make decisions independently of one another while only sharing network-level knowledge. This allows for game termination on simple local goals, where the goal for a lateral movement is to move to any other visible host and the goal for exfiltration is to remove any files from the host the agent resides on. This is in contrast to other approaches where agents must reach a specific end-state on the network [24]. It also reduces the complexity of decision-making for each individual agent on the network, given its scope of action is strictly local.

Within both the emulation and simulation setups there is a distinction between abstract actions and grounded actions. An abstract action is the generic type, such as Copy File. The grounded action populates the action parameters with specific values, such as the path of the file to copy. The reinforcement learning algorithm selects the abstract action. Other mechanisms are then used to select an available grounded action of the provided abstract action type. This two-stage approach allows for the action space of the reinforcement learning algorithm to be small and fixed size. For this experiment, the second stage mechanism is simple random selection.

Given that there are several actions with preconditions on agent knowledge, an action mask is generated based on the current state of the agent's knowledge base and defined constraints on actions. For example, an action to copy a file is constrained by whether the agent has discovered file paths through a scan. The mask is recomputed at each step once a simulation result has been processed and the agent's memory updated. It is then used to clamp the outputs of the fully connected neural network to disallow the selection of constrained actions at that time step. This hastens training by leveraging known constraints to avoid unrealistic policies [20].

The observation space was a simple history of executed actions. The horizon of the action history was set to the total number of actions allowable in an episode and padded to that length with zeros. Actions are appended to the beginning of an action history list and one-hot encoded. The length of the observation vector was equal to the number of available actions (13) multiplied by the horizon of the action history (50) for a total of 650 values.

The reward policy consisted of four components: a small negative reward for a failed action, a small positive reward for gain of new knowledge in the form of facts to encourage exploration, a large constant size positive reward for a successful lateral movement, and a large variable size positive reward for a successful data exfiltration that scales based on how many files were exfiltrated. The total attainable award was equal to approximately one (Table 2).

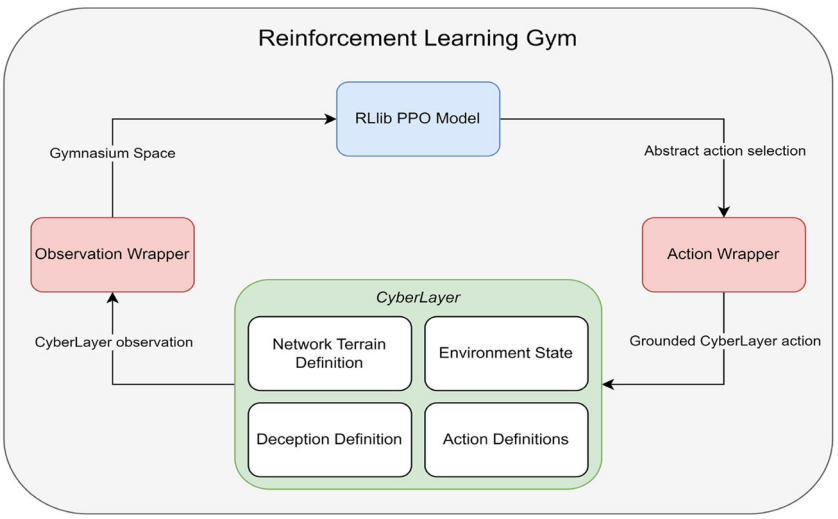

Fig. 3 Data flow within Mirage's reinforcement learning gym

Table 2 Reward values

Reward Component	Value
Failed Action	−0.01
Information Gain	0.0005 per fact
Exfiltration	0.1 per file; 6 files available
Lateral Move	0.4

5.3 Experimental setup

For the purposes of this work, the training algorithm used to update the agent's decision policy was Proximal Policy Optimization (PPO) [36]. There are alternatives including Importance Weighted Actor-Learner Architecture (IMPALA) and other Actor-Critic (AC) algorithms [9, 15]. However, due to PPO's stability, ease of implementation, and the relative speed of CyberLayer, PPO was selected instead [25].

The training was performed on a laptop with a 2.7GHz six-core Intel Core i7 CPU and 32 GB of memory. No GPU was utilized. The training setup was defined as follows:

- Learning Rate — 0.0001
- KL Coefficient — 0.2
- Number of SGD Iterations — 30
- SGD Minibatch Size — 128
- Training Batch Size — 4000
- Maximum steps allowed in episode — 50
- Six parallel workers, with five environments each

The set of actions was designed to match the Thief adversary that was demonstrated in the emulation experiments. To confirm the validity of the actions in CyberLayer, the results of the experiments performed in the emulation environment described in Section 4.2 were used to hand-validate the environment. Implementations of actions in CyberLayer were tuned until results of games performed using an agent that replicates Caldera's planning capabilities closely matched the results obtained from the emulation games (Table 3).

An episode of the game was terminated on three conditions: successful achievement of both exfiltration and lateral movement goals, having no further actions available to the agent, or hitting the episode cap of 50 steps.

The topology of the simulation network used was designed to closely mimic the emulation network described in Section 4.1.4.

5.4 Experimentation results

Figure 4 shows two metrics tracked during training. These metrics demonstrate the algorithm's ability to optimize the rewards it receives from the environment for accomplishing goals and the number of executed actions it took it to do so.

Table 3 Set of actions used in simulation reinforcement learning experimentation

Action Name	Tactic	Technique ID
File Scan	Collection	T1005
Create Directory	Collection	T1074.001
Copy File	Collection	T1074.001
Compress Directory	Exfiltration	T1560.001
Transfer Files	Exfiltration	T1041
Dump Passwords	Credential Access	T1003.001
Get Domain	Discovery	T1482
Get Domain Computers	Discovery	T1018
Get Admin Accounts	Discovery	T1069.002
View Remote Shares	Discovery	T1135
Mount Share	Lateral Movement	T1021.002
Transfer File SMB	Lateral Movement	T1021.002
Execute Remote Binary	Execution	T1047

Through these two metrics, baseline training run with no deception demonstrated that the optimal policy that the algorithm was able to converge on received a higher average reward while performing fewer total actions. Each deception applied had a clear effect on both metrics.

- The Black Hole deception targets a fixed exfiltration staging directory to stop the adversary from copying content to that directory for later exfiltration. It had the effect of stopping the adversary from receiving the bulk of the available reward but also allowed the adversary to prune away that route of attack, resulting in shorter episode lengths.
- The Sneaky Files deception targets an added honey directory to cause files discovered in that directory to have inconsistent filenames. Because of this, an agent's action to copy a file will sometimes result in a failed action. The effect is shown to be both a lower reward from the adversary choosing to exfiltrate less content and an inability to further optimize the attack length.

A secondary result is the demonstration of the efficacy of a simulation environment to better allow for large quantities of games/episodes than the emulation system demonstrated in Section 4.1.4. With the setup described in Section 5.3, each training experiment took an average of 37 minutes and 28 seconds, while executing an average of 5,489 games. This speed of execution over emulation was a key enabler of reinforcement learning and larger-scale assessment of deception against reactive autonomous adversaries.

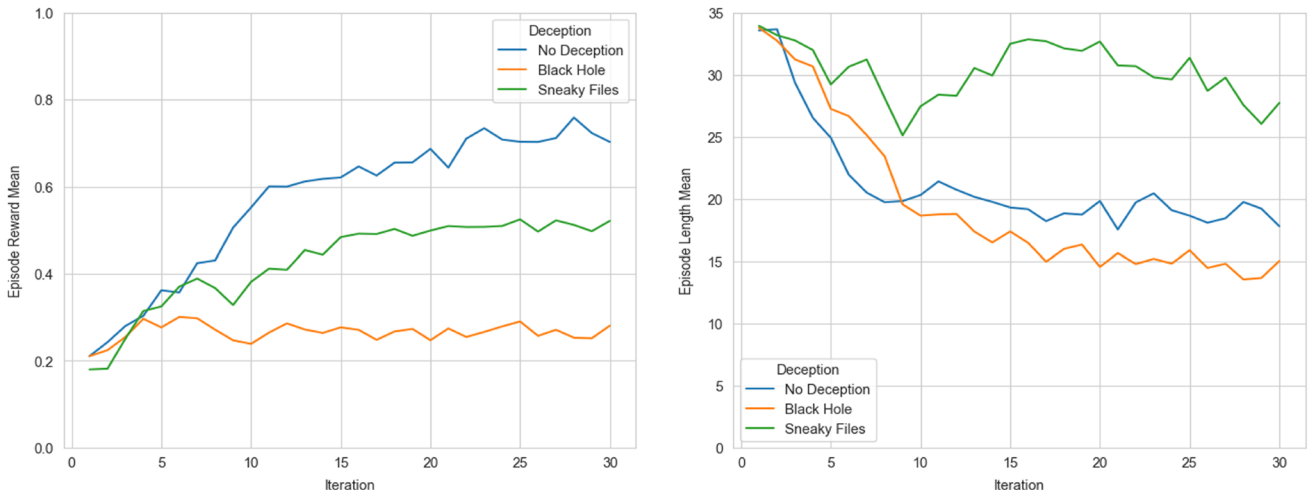

Fig. 4 Mirage simulation experiment results tracking mean reward (left) and mean length of episode (right) during training, with hue denoting type of deception applied

6 Mirage evaluation

The second goal of this work was to evaluate the overall efficacy of the Mirage prototype in capturing and quantifying the deception effects on an autonomous cyber adversary. The following sections discuss the modularity and scalability of Mirage when it comes to evaluating additional novel deceptions and adversary planning algorithms on more complex cyber ranges.

6.1 Modularity of Mirage in emulation

6.1.1 Deceptions and planners

Adding deceptions and planners entails a high development effort and cost. In short, while *Anansi* and Caldera completely allow for new deceptions and planners to be added, each new component requires non-trivial development and testing as the components function on real computer and network systems, often dealing with low-level APIs. For the deceptions under test, the *Anansi* service does allow for standard deployment and actuation (on a Windows host) and thus some efficiency of scale. However, as the deceptions are entirely real, they still require serious test and evaluation.

For the adversarial Caldera planners under test, we make the same conclusion that constant linear development costs are present for any additional planner. Each new Caldera planner must be implemented within Caldera and tested, apriori to any deception experimentation. Additionally, while Caldera serves as a sufficient operational platform for executing adversarial cyber attacks, a known limitation is that Caldera's data model and architecture do restrict the complexity of any implemented planner. For example, the Guided planner we implemented can only guarantee proper execution for the chosen Ransomware and Thief adversary profiles; not all additional adversaries.

6.1.2 Ranges

The Caldera Range plugin allowed Mirage's experimentation program to be deployed on its cyber range with notable automation and ease of repeatability. Cyber range specifications were deployed in a "click-and-shoot" manner. That is, Caldera Range allows for easy specification of desired hosts, services, and network configurations, then automates the building of the cyber range in AWS environment using Ansible and the AWS API. For future experiments, adding network complexity, additional host and network services, user data, etc. would be a relatively low-cost effort. Adding some services would be more complex (e.g., Outlook web server). Compared to creating additional deceptions and planners, the effort required to extend and enhance the cyber range is notably less.

6.1.3 Metrics

Mirage's ability to capture metrics depends on its system sub-components. With Caldera Range as the cyber range component of Mirage, capturing range activity at all levels is amenable. However, there are current limits to capturing internal deliberation metrics from Caldera's planning services, specifically the measurement of state space exploration executed by the Caldera planner. This can be implemented but would have to be done for each Caldera planner under test, as such a mechanism is not built into the core Caldera platform.

6.2 Modularity of Mirage in CyberLayer simulation

6.2.1 Deceptions and agents

In contrast to Mirage's emulation system, adding and encoding deceptions into the CyberLayer system requires significantly less effort. To add deceptions to CyberLayer, one must essentially create a set of (programming) functions that hook any observation feedback sent to the agent, and modify the perceived environment state as required. The data model's API in the CyberLayer backend handles the maintenance of keeping a ground truth instance of the environment. It is also responsible for overlaying instances of deception on the state observed by the agent. For Mirage simulations, the data backend is also very performant as an in-memory network graph was used as the underlying data structure.

For developing and testing additional cyber agents in CyberLayer, there is also a reduction in the required effort; however, the reduction is more nuanced. With the CyberLayer simulation environment and reinforcement learning gym, adding new agents potentially requires implementing additional reward policies, observation spaces, and action mappers. However, the RL algorithms that serve as the underlying decision engine for the agent do not need to be implemented as they already exist in RLlib, and can be applied to cyber agents within CyberLayer simulation environment. This is opposed to our emulation system, where additional cyber agents required additional planners (i.e., the decision engine of the agents in that case) to be implemented outright. For our emulation experiments, observation spaces and action mappers were also abstracted away from Mirage as they are handled implicitly by Caldera's operational API. Overall, the emulation case is more limiting, as in our empirical experience developing new planners for emulations is a higher level of effort than developing new reward policies, observation spaces, and action mappers for simulations.

6.2.2 Cyber environments

With regard to extending and manipulating the cyber environments in CyberLayer, it is naturally also less effort than doing the equivalent in Mirage's emulation system. With CyberLayer, even for more complex actions, systems, services/applications, etc., one is still only creating a very good mock, not the actual implementation, which is always less computational effort. For example, while creating a simulated domain controller is not trivial, it never approaches the cost of creating a real domain controller for an emulated environment. Furthermore, simulated components can be tested and evaluated much more quickly and readily with programmatic tests whereas emulated components necessitate live integration testing.

6.2.3 Metrics

Lastly, as expected with any simulation system, Mirage's use of CyberLayer and RLlib for simulations allows for extensive metrics and performance evaluation. With CyberLayer, the entire state of the cyber environment is accessible, and analytics can be inserted into any aspect of the simulation. Similarly, for the reinforcement learning gym, RLlib, and more specifically Ray, it comes off-the-shelf with significant performance and evaluation capabilities to use in any gym training experiments.

7 Conclusion and future work

In this work, our team initially designed, prototyped, and evaluated Mirage, an experimentation system for evaluating cyber deceptions against autonomous cyber adversaries. Three novel cyber deceptions, tailored to targeting automated planning techniques, were created and evaluated against two types of adversaries (ransomware, data theft). These adversaries were executed by autonomous Caldera planners that utilized different planning techniques (future reward, forward search). To deploy these novel cyber deceptions and allow for their dynamic nature, our team also created *Anansi*, a Windows operating system service to actuate and control the cyber deceptions. To comprehensively emulate and test these novel cyber deceptions and adversaries, we conducted 72 live offensive cyber attacks on an AWS cyber range using Caldera's Range plugin. As a result of these experiments, we assessed three core challenges to expanding on our work: development costs (of emulation), experimentation time (of emulation), and Caldera planner limitations.

To then improve upon Mirage's emulation system, and address those noted challenges, specifically of development cost and experimentation time of emulation, our team proceeded to integrate Mirage into a simulation system, called CyberLayer, to evaluate cyber deceptions at machine speed. CyberLayer enabled high-fidelity simulation of offensive cyber environments where the action spaces in the simulated environment are equivalent to real-world cyber attack actions. Additionally, our team then integrated the reinforcement learning gym RLlib into CyberLayer to enable a full-scale offensive cyber gym to train reinforcement learning agents in simulated cyber environments that contained cyber deceptions.

Our first set of experiments of Mirage with the CyberLayer simulation environment involved a reinforcement learning

cyber agent whose action space was that of the simple Thief cyber adversary, and where the simulated cyber environment had the same computer network architecture and hosts as our previous emulation experiments. The agent was trained with PPO, and the optimum performance of the agent in simulation environments with deception matched the overall trends observed in the previous emulation experiments. That is, as an initial efficacy test, the simulation system proved valid for testing Mirage's cyber deceptions.

Following this work, our team plans to further develop CyberLayer and the integrated RLlib reinforcement learning gym as a means to develop more sophisticated cyber agents that can manage larger action spaces and more complex cyber environments. Specifically, further research and development are required around observation spaces, reward policies, and action mappers in order to support the training of more sophisticated agents. Additionally, as CyberLayer was only quickly introduced in this publication, our team plans to have follow on publications to detail CyberLayer and its reinforcement learning gym at length.

Appendix A. Adversary profiles

Below is the list of abilities used in the Thief and BlackSun adversaries (Table 4). Other than the ransomware abilities marked with a star, all are found in Caldera's stockpile plugin [28] (Table 5).

Table 4 Set of actions used in emulation Thief adversary

Action Name	Tactic	Technique ID
Find Sensitive Files	Collection	T1005
Create Staging Directory	Collection	T1074.001
Stage File	Collection	T1074.001
Compress Staging Directory	Exfiltration	T1560.001
Exfil Compressed Directory	Exfiltration	T1041
Powerkatz (Staged)	Credential Access	T1003.001
Discover local hosts	Discovery	T1018
Find Domain	Discovery	T1016
Discover Domain Admins	Discovery	T1069.002
Account-type Admin Enumerator	Discovery	T1069.002
Remote Host Ping	Discovery	T1016
Mount Share	Lateral Movement	T1021.002
Copy 54ndc47 (SMB)	Lateral Movement	T1021.002
Start 54ndc47 (WMI)	Execution	T1047

Table 5 Set of actions used in emulation BlackSun ransomware adversary

Action Name	Tactic	Technique ID
Disable Windows Defender	Defense Evasion	T1562.001
Stop Backup Services*	Impact	T1489
Delete Shadow Copies*	Impact	T1490
Find Backups*	Discovery	T1083
Overwrite Backup*	Impact	T1490
Find Sensitive Files	Collection	T1005
AESKeyGen*	Resource Development	T1587
Invoke AES*	Impact	T1486
Powerkatz (Staged)	Credential Access	T1003.001
Discover local hosts	Discovery	T1018
Find Domain	Discovery	T1016
Discover Domain Admins	Discovery	T1069.002
Account-type Admin Enumerator	Discovery	T1069.002
Remote Host Ping	Discovery	T1016
Mount Share	Lateral Movement	T1021.002
Copy 54ndc47 (SMB)	Lateral Movement	T1021.002
Start 54ndc47 (WMI)	Execution	T1047
Clear Logs	Defense Evasion	T1070.001

Author contribution MK and RA led the initial writing, experimental questions, and paper layout. DL led experimentation and result collection. DD, ZC, EM, and DL all worked on the emulation capability, building a cyber range, the deceptions, and the experimental framework. GR led the simulation capability creation, contributed to by DL and EM for the deception components, BG and DL for the reinforcement learning integration, RM, EM, and TH for creating the adversary framework and models in the simulation system. All authors reviewed the manuscript.

Funding This research was funded by MITRE's Independent Research and Development Program.
Portions of this technical data were produced for the U. S. Government under Contract No. FA8702-19-C-0001 and W56KGU-18-D-0004, and is subject to the Rights in Technical Data-Noncommercial Items Clause DFARS 252.227-7013 (FEB 2014).
Approved for public release. Distribution unlimited Case: 23-4225 (NSEC MOIE).

Data availability Source code for MITRE Caldera can be found online at https://github.com/mitre/caldera. Access to the BlackSun adversary is currently restricted to avoid releasing Caldera adversaries and TTP's that may cause significant harm. CyberLayer is currently proprietary. Those with interest in the CyberLayer system or experimental datasets can contact caldera@mitre.org.

References

1. Al-Shaer E, Wei J, Kevin W et al (2019) Autonomous cyber deception. Springer
2. Applebaum A, Miller D, Strom B et al (2016) Intelligent, automated red team emulation. In: Proceedings of the 32nd annual conference on computer security applications. Association for Computing Machinery, New York, NY, USA, ACSAC '16, pp 363–373
3. Applebaum A, Miller D, Strom B et al (2017) Analysis of automated adversary emulation techniques. In: Proceedings of the summer simulation multi-conference, pp 1–12
4. Bland JA, Petty MD, Whitaker TS et al (2020) Machine learning cyberattack and defense strategies. Comput Secur 92:101738
5. Brockman G, Cheung V, Pettersson L et al (2016) OpenAI gym. arXiv preprint arXiv:1606.01540
6. Chaudhari P (2022) BlackSun ransomware - the dark side of PowerShell. https://www.blogs.vmware.com/
7. DarkTrace (2020) Study finds AI-fueled attacks are not just sci-fi. https://www.darktrace.com/
8. Dykstra J, Shortridge K, Met J et al (2022) Sludge for good: slowing and imposing costs on cyber attackers. arXiv preprint arXiv:2211.16626
9. Espeholt L, Soyer H, Munos R et al (2018) Impala: scalable distributed deep-RL with importance weighted actor-learner architectures. In: Proceedings of the international conference on machine learning (ICML)
10. Ferguson-Walter K, Shade T, Rogers A et al (2018) The Tularosa study: an experimental design and implementation to quantify the effectiveness of cyber deception. Tech. rep., Sandia National Lab.(SNL-NM), Albuquerque, NM (United States)
11. Ferguson-Walter K, Major M, Johnson CK et al (2021) Examining the efficacy of decoybased and psychological cyber deception. In: USENIX security symposium, pp 1127–1144
12. Ferguson-Walter KJ (2020) An empirical assessment of the effectiveness of deception for cyber defense. PhD thesis, University of Massachusetts Amherst
13. Fraunholz D, Anton SD, Lipps C et al (2018) Demystifying deception technology: a survey. arXiv preprint arXiv:1804.06196
14. Gianvecchio S, Kouremetis M, Applebaum A (2022) Look ahead planner. https://www.medium.com/
15. Grondman I, Busoniu L, Lopes GAD et al (2012) A survey of actor-critic reinforcement learning: standard and natural policy gradients. IEEE Trans Syst Man Cybern Part C Appl Rev 42(6):1291–1307. https://doi.org/10.1109/TSMCC.2012.2218595
16. Guarino A (2013) Autonomous intelligent agents in cyber offence. In: 5th International conference on cyber conflict (CYCON 2013), IEEE, pp 1–12
17. Islam MM, Dutta A, Sajid MSI et al (2021) CHIMERA: autonomous planning and orchestration for malware deception. In: 2021 IEEE Conference on communications and network security (CNS), IEEE, pp 173–181
18. James Waldo KM (2018) Ending the cybersecurity arms race. Belfer Center for Science and International Affairs, Harvard Kennedy School
19. Kaloudi N, Li J (2020) The AI-based cyber threat landscape: a survey. ACM Comput Surv (CSUR) 53(1):1–34
20. Kanervisto A, Scheller C, Hautamäki V (2020) Action space shaping in deep reinforcement learning. In: 2020 IEEE Conference on games (CoG), pp 479–486. https://doi.org/10.1109/CoG47356.2020.9231687
21. Kirat D, Jang J, Stoecklin M (2018) Deeplocker-concealing targeted attacks with AI locksmithing. Blackhat USA 1:1–29
22. Kouremetis M, Alford R, Lawrence D (2023) Mirage: cyber deception against autonomous cyber attacks. In: Proceedings of the 7th cyber security in networking conference (CSNet 2023). IEEE, pp 163–170
23. Lawrence D, Kouremetis M, Applebaum A et al (2022) Guided planner. https://www.medium.com/
24. Li L, Fayad R, Taylor A (2021) CyGIL: A cyber gym for training autonomous agents over emulated network systems. arXiv preprint arXiv:2109.03331
25. Li Y (2018) Deep reinforcement learning: an overview. arXiv:1701.07274
26. Liang E, Liaw R, Nishihara R, et al (2018) RLlib: abstractions for distributed reinforcement learning. In: Proceedings of the 35th international conference on machine learning, vol 80. PMLR, pp 3053–3062
27. Microsoft Defender Research Team (2021) CyberBattleSim. https://github.com/, created by Christian Seifert, Michael Betser, William Blum, James Bono, Kate Farris, Emily Goren, Justin Grana, Kristian Holsheimer, Brandon Marken, Joshua Neil, Nicole Nichols, Jugal Parikh, Haoran Wei
28. MITRE Corporation (2022) MITRE Caldera: a scalable, automated adversary emulation platform. https://www.github.com/
29. MITRE Corporation (2023) MITRE engage: a framework for planning and discussing adversary engagement operations. https://engage.mitre.org/
30. Nawrocki M, Wählisch M, Schmidt TC et al (2016) A survey on honeypot software and data analysis. arXiv preprint arXiv:1608.06249
31. Perlroth N (2021) This is how they tell me the world ends: winner of the FT & McKinsey business book of the year award 2021. Bloomsbury Publishing
32. Pramod B, Beesetty Y, Vineet K (2022) Deception technology market. Tech. Rep. A31357, Allied Market Research
33. Reti D, Fraunholz D, Elzer K et al (2022) Evaluating deception and moving target defense with network attack simulation. In: Proceedings of the 9th ACM workshop on moving target defense, pp 45–53
34. Sajid MSI, Wei J, Alam MR et al (2020) Dodgetron: towards autonomous cyber deception using dynamic hybrid analysis of malware. In: 2020 IEEE Conference on communications and network security (CNS), IEEE, pp 1–9
35. Sajid MSI, Wei J, Abdeen B et al (2021) Soda: a system for cyber deception orchestration and automation. In: Annual computer security applications conference, pp 675–689
36. Schulman J, Wolski F, Dhariwal P et al (2017) Proximal policy optimization algorithms. arXiv:1707.06347
37. Shade T, Rogers A, Ferguson-Walter K et al (2020) The Moonraker study: an experimental evaluation of host-based deception. In: HICSS, pp 1–10
38. Silver D, Schrittwieser J, Simonyan K et al (2017) Mastering the game of go without human knowledge. Nature 550(7676):354–359
39. Spafford E (2011) More than passive defense. https://www.cerias.purdue.edu/
40. Standen M, Lucas M, Bowman D et al (2021) CybORG: a gym for the development of autonomous cyber agents. arXiv preprint arXiv:2108.09118
41. Truong TC, Plucar J, Diep BQ et al (2022) X-ware: a proof of concept malware utilizing artificial intelligence. Int J Electr Comput Eng (IJECE) 12(2):1937–1944

42. Walter E, Ferguson-Walter K, Ridley A (2021) Incorporating deception into Cyber- BattleSim for autonomous defense. arXiv preprint arXiv:2108.13980
43. Wang C, Lu Z (2018) Cyber deception: overview and the road ahead. IEEE Secur Priv 16(2):80–85. https://doi.org/10.1109/MSP.2018.1870866
44. Wang Z, Zhang Y, Liu Z et al (2021) An automatic planning-based attack path discovery approach from IT to OT networks. Secur Commun Netw 2021:1–18
45. Zhang L, Thing VL (2021) Three decades of deception techniques in active cyber defenseretrospect and outlook. Comput Secur 106:102288